Sorting dan Searching

No 2

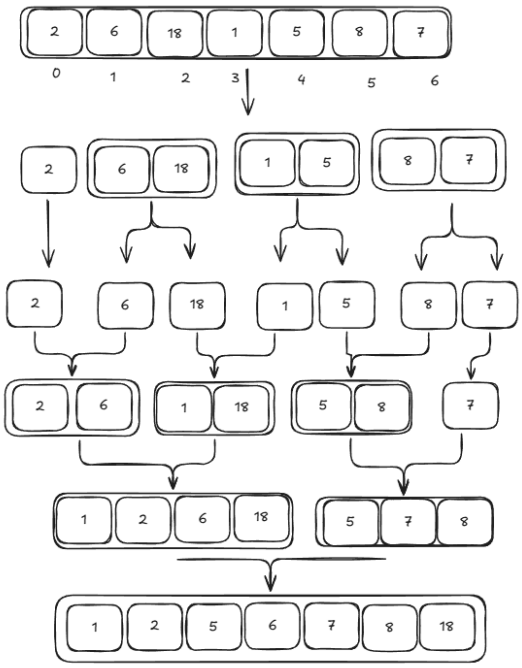
Struktur Data

PENGERTIAN SORTING

Sorting merupakan Pengurutan data dalam struktur data sangat penting terutama untuk data yang bertipe data numerik ataupun karakter. Pengurutan dapat dilakukan secara ascending (dari yang terkecil ke terbesar) dan descending (dari yang terbesar ke terkecil). Pengurutan (Sorting) adalah proses pengurutan data yang sebelumnya disusun secara acak sehingga tersusun secara teratur menurut aturan tertentu. Sama seperti halnya metode metode lain yang ada pada pemrograman, sorting juga tidak lepas kaitannya dengan kompleksitas waktu atau efiensi program, oleh karena itu penting untuk memilih metode mana yang terbaik. Dalam pengurutan data terdapat berbagai macam metode, beberapa diantaranya yaitu

1. Merge Sort

   Merupakan salah satu teknik sorting yang menggunakan algoritma Divide and Conquer yang dimana cara kerja dari teknik yaitu memecah kumpulan array sehingga semua terpisah yang selanjutnya akan diurutkan dan digabungkan kembali. Berikut adalah ilustrasi dari merge sort:



Gambar 1 Ilustrasi Merge Sort

```
#include <iostream>
#include <string>

using namespace std;

// Definisi Struct
struct Produk {
    int id;
    string nama;
    int harga;
};

// Fungsi untuk menggabungkan dua bagian array Produk
void merge(Produk* arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Membuat array sementara untuk menampung struct
    Produk* L = new Produk[n1];
    Produk* R = new Produk[n2];

    // Menyalin data ke array sementara menggunakan pointer akses
    for (int i = 0; i < n1; i++)
        *(L + i) = *(arr + left + i);
    for (int j = 0; j < n2; j++)
        *(R + j) = *(arr + mid + 1 + j);

    int i = 0, j = 0, k = left;

    // Proses penggabungan berdasarkan property 'harga'
    while (i < n1 && j < n2) {
        if ((L + i)->harga <= (R + j)->harga) {
            *(arr + k) = *(L + i);
            i++;
        } else {
            *(arr + k) = *(R + j);
            j++;
        }
        k++;
    }
```

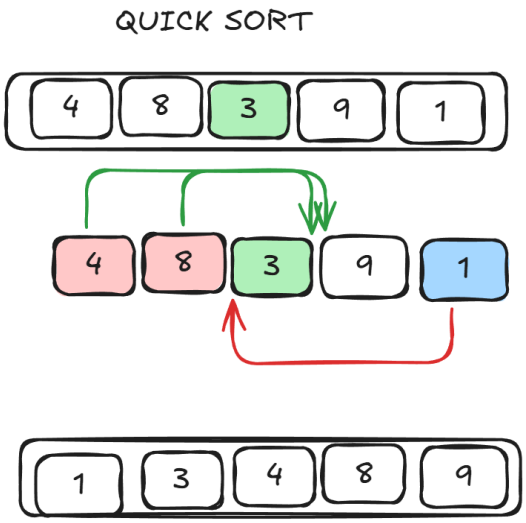
```
    // Menyalin sisa elemen
    while (i < n1) {
        *(arr + k) = *(L + i);
        i++; k++;
    }
    while (j < n2) {
        *(arr + k) = *(R + j);
        j++; k++;
    }

    delete[] L;
    delete[] R;
}

// Fungsi Rekursif Merge Sort
void mergeSort(Produk* arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

// Fungsi untuk menampilkan isi Array of Struct
void tampilkanData(Produk* arr, int n) {
    cout << "ID\tNama\tHarga" << endl;
    cout << "------------------------" << endl;
    for (int i = 0; i < n; i++) {
        // Mengakses anggota struct menggunakan pointer (ptr->member)
        Produk* ptr = (arr + i);
        cout << ptr->id << "\t" << ptr->nama << "\t" << ptr->harga << endl;
    }
    cout << endl;
}

int main() {
```

```
    int n = 4;
    // Inisialisasi Array of Struct
    Produk daftarProduk[4] = {
        {101, "Laptop", 8000},
        {102, "Mouse ", 150},
        {103, "Monitor", 2500},
        {104, "Keyboard", 500}
    };

    cout << "Data Sebelum Sorting (Berdasarkan Harga):" << endl;
    tampilkanData(daftarProduk, n);

    // Memanggil Merge Sort menggunakan pointer ke array
    mergeSort(daftarProduk, 0, n - 1);

    cout << "Data Setelah Sorting (Merge Sort O(n log n)):" << endl;
    tampilkanData(daftarProduk, n);

    return 0;
}
```

Merge sort selalu memiliki kompleksitas O(n log n) dikarenakan sekalipun data yang paling acak diberikan kepada metode/algoritma ini merge sort akan tetap membagi 2 data sampai akhirnya menjadi data tunggal yang akan kembali digabungkan berdasarkan tipe penyortiran (ascending/descending).

2. Quick Sort

Sama seperti Merge Sort dimana teknik ini juga menggunakan algoritma divide and conquer dimana cara kerja dari teknik yaitu dengan mengambil salah satu elemen yang akan disebut sebagai pivot dan mempartisi (membagi) array yang berada di sekitar pivot, lalu akan dilakukan pengurutan berdasarkan perbandingan elemen yang lebih kecil atau lebih besar dari pivot. Berikut adalah ilustrasi dari quick sort:




```
#include <iostream>
#include <string>

using namespace std;

// Definisi Struct Mahasiswa
struct Mahasiswa {
    string nim;
    string nama;
};

// Fungsi untuk menukar dua objek Mahasiswa menggunakan pointer
void swap(Mahasiswa* a, Mahasiswa* b) {
    Mahasiswa temp = *a;
    *a = *b;
    *b = temp;
```

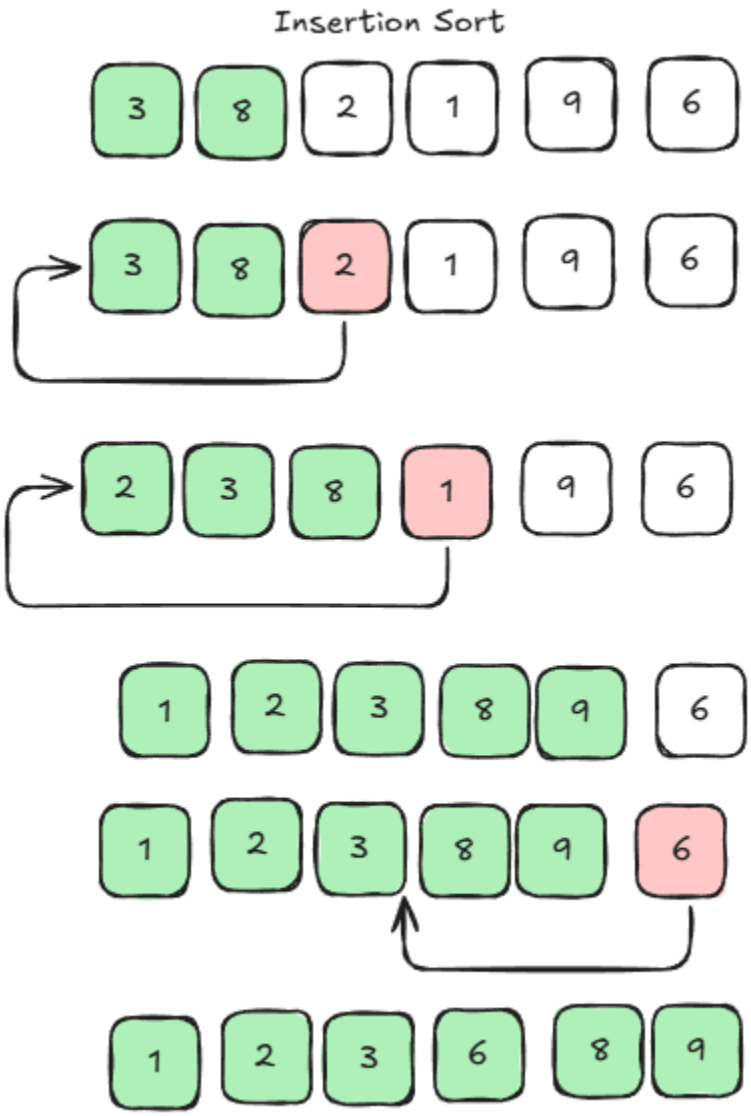
```
}

// Fungsi Partition: Inti dari Quick Sort
int partition(Mahasiswa* arr, int low, int high) {
    // Memilih elemen terakhir sebagai pivot
    string pivot = (arr + high)->nim;
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) {
        // Jika NIM saat ini lebih kecil dari pivot
        if ((arr + j)->nim < pivot) {
            i++;
            swap((arr + i), (arr + j));
        }
    }
    swap((arr + i + 1), (arr + high));
    return (i + 1);
}

// Fungsi Utama Quick Sort
void quickSort(Mahasiswa* arr, int low, int high) {
    if (low < high) {
        // pi adalah index partisi, arr[pi] sudah di posisi yang benar
        int pi = partition(arr, low, high);

        // Sort elemen secara terpisah sebelum dan sesudah partisi
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

void tampilkanData(Mahasiswa* arr, int n) {
    cout << "NIM\t\tNama" << endl;
    cout << "----------------------------" << endl;
    for (int i = 0; i < n; i++) {
        cout << (arr + i)->nim << "\t" << (arr + i)->nama << endl;
    }
    cout << endl;
}
```

```
int main() {
    int n = 5;
    Mahasiswa kelas[] = {
        {"220910", "Budi"},
        {"220901", "Ani"},
        {"220915", "Citra"},
        {"220905", "Dodi"},
        {"220912", "Eka"}
    };

    cout << "=== DATA MAHASISWA (SEBELUM SORT) ===" << endl;
    tampilkanData(kelas, n);

    quickSort(kelas, 0, n - 1);

    cout << "=== DATA MAHASISWA (SESUDAH QUICK SORT) ===" << endl;
    tampilkanData(kelas, n);

    return 0;
}
```

Jika membahas mengenai quick sort tentunya kita akan mengetahui bahwa quicksort merupakan salah satu algoritma tercepat apabila pivot yang digunakan merupakan nilai tengah data, tetapi apabila data sudah terurut dan memilih pivot yang paling ujung, hal ini akan memberikan word case yaitu kompleksitas waktu $O(n^2)$

3. Insertion Sort

Insertion Sort merupakan sebuah teknik pengurutan dengan cara membandingkan dan mengurutkan dua data pertama pada array, kemudian membandingkan data para array berikutnya apakah sudah berada di tempat semestinya. Algorithma insertion sort seperti proses pengurutan kartu yang berada di tangan kita. Algorithma ini dapat mengurutkan data dari besar ke kecil (Ascending) dan kecil ke besar (Descending).



```
#include <iostream>
#include <string>

using namespace std;

// Definisi Struktur Data Buku
struct Buku {
    int idBuku;
    string judul;
};

// Fungsi Insertion Sort dengan akses Pointer
void insertionSort(Buku* rak, int n) {
    // Dimulai dari buku kedua (index 1)
    for (int i = 1; i < n; i++) {
        // 'key' adalah buku yang sedang diambil dari rak untuk disisipkan
        Buku key = *(rak + i);
        int j = i - 1;

        /* Loop While: Mencari posisi yang tepat untuk 'key'.
           Di sini terjadi perbandingan (Comparison) dan pergeseran
(Shifting).
        */
        while (j >= 0 && (rak + j)->idBuku > key.idBuku) {
            // Geser buku yang lebih besar ke kanan
            *(rak + j + 1) = *(rak + j);
            j = j - 1;
        }
        // Letakkan buku 'key' di posisi yang sudah kosong
        *(rak + j + 1) = key;
    }
}

// Fungsi untuk menampilkan daftar buku di rak
void tampilkanRak(Buku* rak, int n) {
    cout << "ID\t| Judul Buku" << endl;
    cout << "-------------------------" << endl;
    for (int i = 0; i < n; i++) {
        cout << (rak + i)->idBuku << "\t| " << (rak + i)->judul << endl;
    }
```

```
    cout << endl;
}

int main() {
    int n = 5;

    // --- SKENARIO 1: BEST CASE ---
    // Buku sudah terurut dari ID terkecil ke terbesar.
    Buku rakUrut[] = {
        {101, "Laskar Pelangi"},
        {102, "Bumi Manusia"},
        {103, "Negeri 5 Menara"},
        {104, "Filosofi Teras"},
        {105, "Hujan"}
    };

    cout << ">>> SKENARIO: BEST CASE (Sudah Terurut) <<<" << endl;
    cout << "Data Sebelum: (Sudah rapi)" << endl;

    insertionSort(rakUrut, n);

    tampilkanRak(rakUrut, n);
    cout << "Analisis: Loop 'while' tidak pernah berjalan. Hanya perlu n
iterasi." << endl;
    cout << "Kompleksitas Waktu: O(n)" << endl;
    cout << "----------------------------------------" << endl << endl;


    // --- SKENARIO 2: WORST CASE ---
    // Buku terurut terbalik (ID terbesar di depan).
    Buku rakTerbalik[] = {
        {105, "Hujan"},
        {104, "Filosofi Teras"},
        {103, "Negeri 5 Menara"},
        {102, "Bumi Manusia"},
        {101, "Laskar Pelangi"}
    };

    cout << ">>> SKENARIO: WORST CASE (Terurut Terbalik) <<<" << endl;
    cout << "Data Sebelum:" << endl;
```

```
    tampilkanRak(rakTerbalik, n);



    insertionSort(rakTerbalik, n);

    cout << "Data Sesudah (Berhasil Diurutkan):" << endl;
    tampilkanRak(rakTerbalik, n);
    cout << "Analisis: Setiap buku harus dibandingkan dan digeser ke posisi
paling awal." << endl;
    cout << "Kompleksitas Waktu: O(n^2)" << endl;

    return 0;
}
```

STUDI KASUS

1. Budi sedang menunggu bis di terminal dengan bengan pertimbangan harga untuk menumpang bis dengan list bis dan harga sebagai berikut:
   - Go jaya: rp 9000
   - Marakana: 8500
   - Suksessdor: 12000
   - Morganda: 11000

   Untuk membantu Budi memilih list harga terbaik, tolong urutkan dengan berbagai macam tipe sortinng dengan pegurutan ascending, gunakan pula perbandingan wort case dan bsae case yang mungkin terjadi pada setiap metode yang digunakan!